

## Practical No: 11

### Code:

```
import java.io.IOException;
import java.util.StringTokenizer;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

    // Mapper Class
    // Input: Line of text
    // Output: <word, 1>
    public static class TokenizerMapper
        extends Mapper<Object, Text, Text, IntWritable> {

        private final static IntWritable one = new
IntWritable(1); // constant value for count
        private Text word = new Text(); // holds each word

        public void map(Object key, Text value, Context
context
        ) throws IOException, InterruptedException {
            // Tokenize the line, convert to lowercase and
remove punctuation
            StringTokenizer itr = new
StringTokenizer(value.toString().toLowerCase().replaceAll("[^a
-zA-Z ]", ""));
            while (itr.hasMoreTokens()) {
                word.set(itr.nextToken()); // get next word
                context.write(word, one); // emit <word, 1>
            }
        }

        // Reducer Class
        // Input: <word, list of counts>
        // Output: <word, total count>
        public static class IntSumReducer
            extends Reducer<Text, IntWritable, Text,
IntWritable> {

            private IntWritable result = new IntWritable();
```

```

        public void reduce(Text key, Iterable<IntWritable>
values,
                                Context context
        ) throws IOException, InterruptedException {
            int sum = 0;
            for (IntWritable val : values) {
                sum += val.get(); // sum all counts for the
word
            }
            result.set(sum);
            context.write(key, result); // emit <word, total>
        }
    }

    // Driver Code
    // Sets up and configures the job
    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.err.println("Usage: WordCount <input path>
<output path>");
            System.exit(-1);
        }

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "word count");

        job.setJarByClass(WordCount.class); // specify class
containing main()
        job.setMapperClass(TokenizerMapper.class); // set
mapper class
        job.setCombinerClass(IntSumReducer.class); // optional
combiner
        job.setReducerClass(IntSumReducer.class); // set
reducer class

        job.setOutputKeyClass(Text.class); // output key type
        job.setOutputValueClass(IntWritable.class); // output
value type

        FileInputFormat.addInputPath(job, new Path(args[0]));
// input path
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
// output path

        // Submit the job and wait
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}

```

**Output:**

|             |   |
|-------------|---|
| a           | 1 |
| cool        | 1 |
| fast        | 1 |
| hadoop      | 2 |
| is          | 3 |
| mapreduce   | 1 |
| model       | 1 |
| programming | 1 |

## Practical No: 12

### Code:

#### 1. Mapper Code – SalesMapper.java:

```
package SalesCountry;

import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.*;

public class SalesMapper extends MapReduceBase implements
Mapper<LongWritable, Text, Text, IntWritable> {

    private final static IntWritable one = new IntWritable(1);

    public void map(LongWritable key, Text value,
OutputCollector<Text, IntWritable> output, Reporter reporter)
        throws IOException {
        String valueString = value.toString();
        String[] fields = valueString.split(",");

        // Check for valid data
        if (fields.length > 7) {
            output.collect(new Text(fields[7].trim()), one);
        }
    }
}
```

#### 2. Reducer Class – SalesCountryReducer:

```
package SalesCountry;

import java.io.IOException;
import java.util.Iterator;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.*;

public class SalesCountryReducer extends MapReduceBase
implements Reducer<Text, IntWritable, Text, IntWritable> {

    public void reduce(Text key, Iterator<IntWritable> values,
OutputCollector<Text, IntWritable> output,
        Reporter reporter) throws IOException {

        int total = 0;

        while (values.hasNext()) {
```

```

        total += values.next().get();
    }

    output.collect(key, new IntWritable(total));
}
}

```

### 3. Driver Class – SalesCountryDriver:

```

package SalesCountry;

import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapred.*;

public class SalesCountryDriver {

    public static void main(String[] args) {
        JobClient client = new JobClient();
        JobConf conf = new JobConf(SalesCountryDriver.class);

        conf.setJobName("SalePerCountry");

        conf.setOutputKeyClass(Text.class);
        conf.setOutputValueClass(IntWritable.class);

        conf.setMapperClass(SalesCountry.SalesMapper.class);

        conf.setReducerClass(SalesCountry.SalesCountryReducer.class);

        conf.setInputFormat(TextInputFormat.class);
        conf.setOutputFormat(TextOutputFormat.class);

        FileInputFormat.setInputPaths(conf, new Path(args[0]));
        FileOutputFormat.setOutputPath(conf, new
Path(args[1]));

        client.setConf(conf);

        try {
            JobClient.runJob(conf);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

#### Sample Input (sales.csv)

```

1,2024-04-20,ProductA,100,John,Doe,India,Pune
2,2024-04-21,ProductB,150,Jane,Smith,India,Mumbai
3,2024-04-22,ProductC,200,Ali,Khan,India,Nashik
4,2024-04-23,ProductD,130,Ravi,Patel,India,Pune
5,2024-04-24,ProductE,170,Sara,Jain,India,Nashik
6,2024-04-25,ProductF,140,Mike,Lee,India,Kolhapur

```

**Output:**

|          |   |
|----------|---|
| Kolhapur | 1 |
| Mumbai   | 1 |
| Nashik   | 2 |
| Pune     | 2 |

## Practical No: 13

### Code:

#### ▪ WordCount.scala

```
import org.apache.spark.sql.SparkSession

object WordCount {
  def main(args: Array[String]): Unit = {
    // Create SparkSession
    val spark = SparkSession.builder
      .appName("Simple WordCount")
      .master("local[*]") // Run locally with all cores
      .getOrCreate()

    // Read the input file
    val textFile = spark.sparkContext.textFile("input.txt")

    // Split each line into words and count occurrences
    val wordCounts = textFile
      .flatMap(line => line.split(" ")) // Split by
space
      .map(word => (word.toLowerCase(), 1)) // Convert
to lowercase and pair with 1
      .reduceByKey(_ + _) // Sum
values for each word

    // Print the result
    wordCounts.collect().foreach(println)

    // Stop SparkSession
    spark.stop()
  }
}
```

### Output:

```
(spark,2)
(is,3)
(fast,1)
(fun,1)
(big,1)
(data,1)
(powerful,1)
```